DS-3003: DWBI Assignment #3

Question 1: Optimizing Multi-table Join

1. Optimized Query (MySQL)

The following query joins Students, Enrollments, Courses, Departments, and Grades to extract the required academic record.

```
select
    concat(s.FirstName, ' ', s.LastName) as StudentName,
    d.DepartmentName,
    c.CourseName,
    c.Credits,
    g.GradeValue
from
    Students s
    join Enrollments e on s.StudentID = e.StudentID
    join Courses c on e.CourseID = c.CourseID
    join Departments d on c.DepartmentID = d.DepartmentID
    join Grades g on e.EnrollmentID = g.EnrollmentID;
```

2. Optimization Decisions

A. Indexing Strategies

To enhance join efficiency:

- Foreign key Indexing: Ensure standard indices exist on Courses.DepartmentID and Grades.EnrollmentID.
- Covering Index (Composite): Implement a composite index on Enrollments(StudentID, CourseID, EnrollmentID). This allows the query execution engine to satisfy the join requirements using only the index structure (Index Scan) rather than accessing the table heap, significantly reducing I/O latency.

B. Materialized Views

- Recommendation: Implement a Materialized View for "Historical Transcripts".
- Justification: Grade data for previous semesters is immutable. Re-calculating a 5-table join for historical data is computationally expensive and unnecessary. A materialized view pre-computes and physically stores this dataset, allowing for near-instant retrieval. The view would be refreshed only when new grades are finalized.

C. Partitioning

- Recommendation: Horizontal Range Partitioning on Enrollments and Grades by Year.
- Justification: Queries predominantly target current students. Storing historical data (e.g., 2015-2020) in the same active partition as current data (2025) increases index depth and search time. Partitioning allows the engine to prune old data ("Partition Pruning") during current-semester queries.

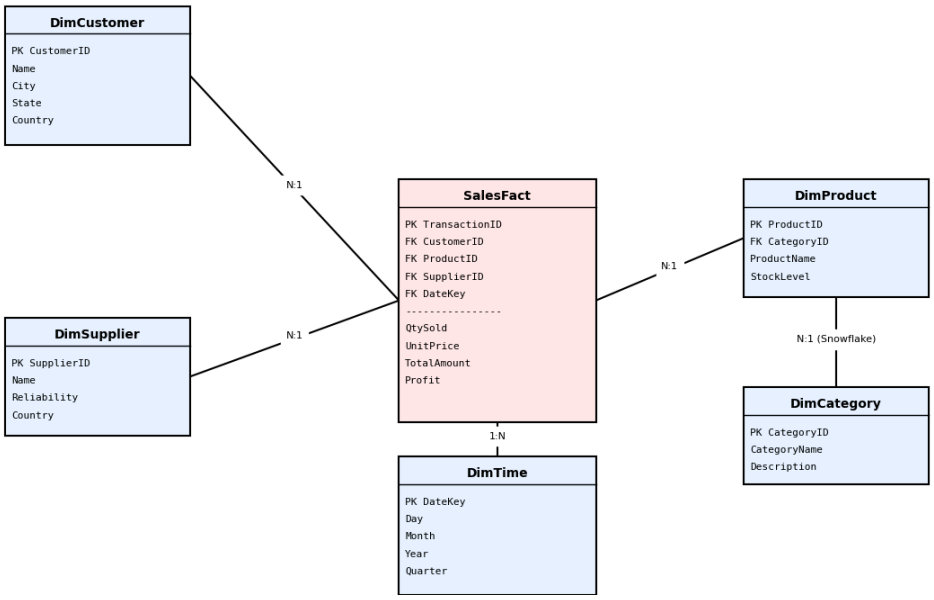
Question 2: Starflake Schema (Retail Analytics)

Scenario: EcoMart Retail Warehouse

1. Schema Structure

- Fact table: SalesFact
    - Granularity: Transaction line level.
    - Measures: QuantitySold, UnitPrice, TotalAmount, Profit .
- Star Dimensions (Denormalized):
    - DimCustomer: Contains City, State, Country directly to avoid joins.
    - DimSupplier: Contains location and reliability ratings directly.
- Snowflake Dimension (Normalized):
    - DimProduct: Normalized into DimProduct -> DimCategory to handle product hierarchy efficiently.

2. Schema Diagram (MERD)

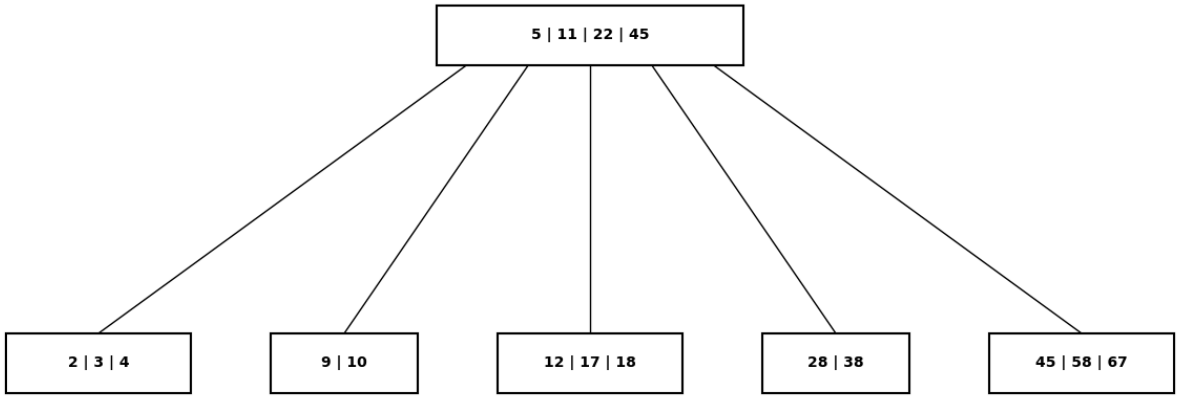


Question 3: Tree Structures

Dataset: {10, 9, 4, 3, 22, 28, 5, 2, 17, 11, 18, 45, 67, 45, 58, 12, 38}.
Sorted: 2, 3, 4, 5, 9, 10, 11, 12, 17, 18, 22, 28, 38, 45, 45, 58, 67.

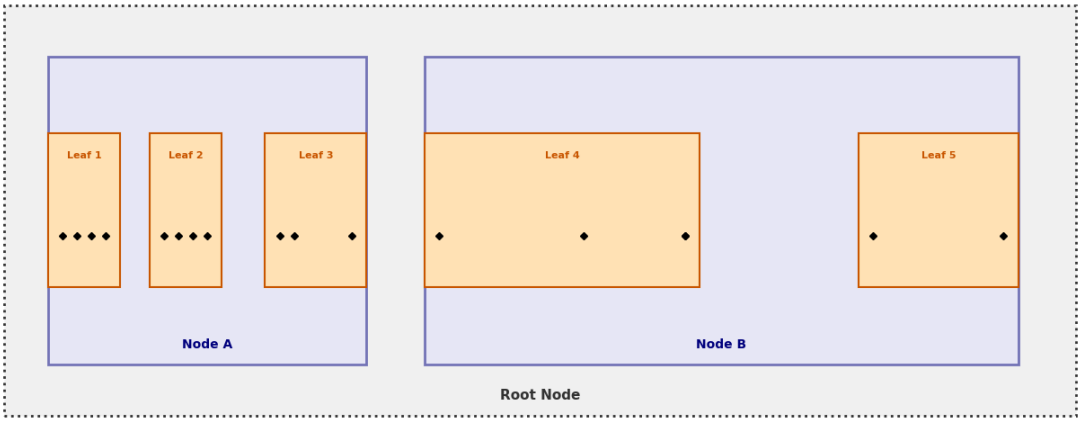
1. 5-Way B-Tree

- Parameters: Order m=5. Max keys = 4. Min keys = 2.
- Structure: The tree is balanced, with the root splitting as it fills beyond capacity (4 keys), promoting median values (e.g., 4, 10, 17, 38) to parent nodes.



2. R-Tree with MBR (Minimum Bounding Range)

- Logic: Since the data is 1-dimensional, the Minimum Bounding Rectangles (MBRs) act as intervals grouping sorted numbers into clusters (Leaves) which are then grouped by parent nodes.



Question 4: Partitioning

Scenario: Sales Transactions dataset (Jan 2020 - Jan 2022).

Task 1: Vertical Partitioning

Strategy: Split the table based on column access frequency.

- Frequently Used: TransactionID, TransactionDate, TotalAmount, Region (Used for BI aggregation).
- Seldom Used: Quantity, UnitPrice, Discount, CategoryID (Used for deep-dive auditing).

SQL Implementation:

```
-- create Frequent table
create table FrequentlyUsedData
(
   TransactionID INT PRIMARY KEY,
   ProductID INT,
   CustomerID INT,
   TransactionDate DATETIME,
   TotalAmount DECIMAL(10, 2),
   Region VARCHAR(50)
);

-- create Seldom Used table
create table SeldomUsedData
(
   TransactionID INT PRIMARY KEY,
   CategoryID INT,
   Quantity INT,
   UnitPrice DECIMAL(10, 2),
   Discount DECIMAL(5, 2),
   foreign key (TransactionID) references FrequentlyUsedData(TransactionID)
);

-- Transfer Data
insert into FrequentlyUsedData
select TransactionID, ProductID, CustomerID, TransactionDate, TotalAmount, Region
from SalesTransactions;

insert into SeldomUsedData
select TransactionID, CategoryID, Quantity, UnitPrice, Discount
from SalesTransactions;
```

Task 2: Horizontal Partitioning

Strategy: Range Partitioning based on TransactionDate.

- Partition 1 (Historical): 2020 Data.
- Partition 2 (Recent): 2021-2022 Data.

SQL Implementation:

```
-- Partition 1: 2020 Data
create table Sales_Part_2020
(
   TransactionID INT,
   TransactionDate DATETIME,
   PRIMARY KEY (TransactionID, TransactionDate)
);

-- Partition 2: 2021-2022 Data
create table Sales_Part_2021_2022
(
   TransactionID INT,
   TransactionDate DATETIME,
   PRIMARY KEY (TransactionID, TransactionDate)
);

-- Transfer Data Logic
insert into Sales_Part_2020
select * from SalesTransactions
where TransactionDate >= '2020-01-01' and TransactionDate < '2021-01-01';

insert into Sales_Part_2021_2022
select * from SalesTransactions
where TransactionDate >= '2021-01-01' and TransactionDate <= '2022-01-31';
```